



Universidad Nacional
Autónoma de México

Facultad de Ciencias
Ciencias de la Computación



Práctica 1

De Regex a NFA

Laboratorio de Compiladores
Semestre 2026-2

Alumno:
Diego Eduardo Peña Villegas
No. Cuenta: 321260682

Fecha de entrega:
10 de marzo de 2026

1. Validación

El enlace al video de validación es el siguiente:

<https://drive.google.com/drive/folders/1ej0jFjj2HeVfPkUV3XMPGyz-L6oyM3TM?usp=sharing>

2. Descripción de la Implementación

2.1. Parser de Expresiones Regulares

Se implementó un parser que realiza dos transformaciones principales:

1. Conversión de concatenación implícita a explícita.
2. Conversión de notación infija a postfija mediante el algoritmo de Shunting-Yard.

La concatenación implícita se detecta cuando un átomo termina y el siguiente inicia sin operador explícito. En estos casos se inserta el operador '.'.

El algoritmo de Shunting-Yard se utilizó para manejar precedencias:

- Operadores unarios: *, +, ?
- Concatenación explícita: .
- Alternación: |

2.2. Construcción del NFA (Algoritmo de Thompson)

Se implementó el algoritmo de Thompson utilizando una pila de fragmentos.

Cada fragmento contiene:

- Estado inicial
- Estado de aceptación

Se implementaron reglas para:

- Literal
- Concatenación
- Alternación
- Kleene Star (*)
- Uno o más (+)
- Cero o uno (?)

Cada operador genera nuevos estados y transiciones ϵ según la definición formal del algoritmo.

2.3. Simulación del NFA

La simulación se realizó mediante:

1. Cálculo de la ϵ -closure.
2. Movimiento por símbolo.
3. Nueva ϵ -closure.

Se utilizó un arreglo dinámico de estados activos para representar el conjunto actual durante la simulación.

Una cadena es aceptada si el estado de aceptación pertenece al conjunto final de estados activos.

3. Dificultades Encontradas

3.1. Manejo de operadores unarios (+ y ?)

Inicialmente, los operadores '+' y '?' eran tratados como literales, lo que generaba errores en el validador. Se corrigió:

- Ajustando el parser para reconocerlos como operadores unarios.
- Extendiendo Thompson para soportar ambos cuantificadores.

3.2. Inserción correcta de concatenación explícita

Fue necesario definir cuidadosamente cuándo insertar el operador '.' para evitar errores de precedencia.

3.3. Simulación eficiente del NFA

Se implementó correctamente el cálculo de ϵ -closure utilizando un stack para evitar ciclos infinitos.

4. Resultados Obtenidos

Se ejecutó el contenedor Docker proporcionado para la validación automática del proyecto mediante el siguiente comando:

```
docker build -t regex_to_nfa_validator .  
docker run --rm regex_to_nfa_validator
```

A continuación se muestra la captura de pantalla de la ejecución del contenedor:

```
diego@lightcore: ~/Practicas 2026-2/Compiladores/regex_to_nfa$ docker run --rm regex_to_nfa_validator
-- The C compiler identification is GNU 15.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Configuring done (0.1s)
-- Generating done (0.0s)
-- Build files have been written to: /app
[ 25%] Building C object CMakeFiles/regex_to_nfa.dir/src/main.c.o
[ 50%] Building C object CMakeFiles/regex_to_nfa.dir/src/nfa.c.o
[ 75%] Building C object CMakeFiles/regex_to_nfa.dir/src/regex.c.o
[100%] Linking C executable regex_to_nfa
[100%] Built target regex_to_nfa
Regex: a|ba*b(a|b)*
  postfix ok
  nfa ok: 11100
Regex: a(b|c)*d
  postfix ok
  nfa ok: 1110
Regex: (a|b)*c
  postfix ok
  nfa ok: 110
Regex: ab
  postfix ok
  nfa ok: 100
Regex: a|b|c
  postfix ok
  nfa ok: 1110
Regex: a*
  postfix ok
  nfa ok: 1110
Regex: ab*c
  postfix ok
  nfa ok: 110
Regex: a|b
  postfix ok
  nfa ok: 110
Regex: (ab)*
  postfix ok
  nfa ok: 1110
Regex: (a|b)*c*d
  postfix ok
  nfa ok: 1110
Overall: PASS
diego@lightcore: ~/Practicas 2026-2/Compiladores/regex_to_nfa$
```

Esto confirma que:

- La conversión a notación postfija es correcta.
- La construcción del NFA mediante Thompson es correcta.
- La simulación del NFA produce los resultados esperados.

5. Análisis de Complejidad

En esta sección se analiza la complejidad temporal y espacial de los principales componentes del sistema implementado.

5.1. Construcción del NFA

Sea n la longitud de la expresión regular de entrada.

1. Inserción de concatenación explícita

El proceso de conversión de concatenación implícita a explícita recorre la expresión carácter por carácter una sola vez, evaluando condiciones constantes para determinar si debe insertarse el operador '.'.

Por lo tanto, este proceso tiene complejidad:

$$O(n)$$

ya que cada símbolo se analiza una única vez.

2. Conversión a notación postfija (Shunting-Yard)

El algoritmo de Shunting-Yard también procesa cada símbolo exactamente una vez.

Cada operador puede ser insertado y removido de la pila a lo sumo una vez, lo que garantiza que el número total de operaciones es lineal respecto al tamaño de la entrada.

Por lo tanto, la complejidad temporal es:

$$O(n)$$

y la complejidad espacial es también $O(n)$ debido al uso de la pila de operadores.

3. Construcción del NFA (Algoritmo de Thompson)

Durante la construcción del NFA:

- Cada símbolo literal genera dos estados y una transición.
- Cada operador genera una cantidad constante de estados y transiciones.

Dado que el número de operadores y literales es proporcional a n , el número total de estados creados es $O(n)$.

Asimismo, el número total de transiciones también es proporcional a n .

Por lo tanto, la complejidad temporal de la construcción es:

$$O(n)$$

y la complejidad espacial del NFA resultante también es:

$$O(n)$$

La construcción completa del NFA (incluyendo parsing y Thompson) tiene complejidad lineal en el tamaño de la expresión regular.

$$O(n)$$

5.2. Simulación del NFA

Sea:

- m la longitud de la cadena de entrada.
- k el número de estados del NFA.

1. Cálculo de ϵ -closure

El cálculo de la ϵ -closure implica explorar las transiciones ϵ alcanzables desde un conjunto de estados.

En el peor caso, puede ser necesario recorrer todos los estados y transiciones ϵ , lo que tiene costo:

$$O(k)$$

por cada cálculo de cierre.

2. Transición por símbolo

Para cada símbolo de la cadena:

- Se recorren los estados activos.
- Se examinan sus transiciones salientes.

En el peor caso, el conjunto activo puede contener hasta k estados.

Por lo tanto, el costo por símbolo es:

$$O(k)$$

3. Complejidad total de simulación

Dado que el proceso anterior se repite para cada uno de los m símbolos de la cadena, la complejidad total es:

$$O(m \cdot k)$$

En el peor escenario teórico, si cada estado tuviera múltiples transiciones ϵ encadenadas y se recalculara repetidamente el cierre, el comportamiento podría acercarse a:

$$O(m \cdot k^2)$$

Sin embargo, en la implementación práctica realizada, el uso de estructuras auxiliares para marcar estados visitados evita exploraciones redundantes, manteniendo el comportamiento cercano a $O(m \cdot k)$.

5.3. Resumen General

- Construcción del NFA: $O(n)$
- Espacio del NFA: $O(n)$
- Simulación: $O(m \cdot k)$

Esto demuestra que la solución implementada es eficiente y escalable para expresiones regulares de tamaño moderado.

6. Conclusión

Se implementó exitosamente un sistema completo de:

- Conversión de expresiones regulares.
- Construcción de NFA mediante Thompson.
- Simulación eficiente del autómata.

La solución cumple completamente con los requerimientos establecidos en el manual del laboratorio y supera todos los casos del sistema de validación automático.